

Introducere în programare - curs și laborator - suport electronic

(C) 2023-2024 Bogdan Pătruț

Meniu

[Lectia 1](#) | [Lectia 2](#) | [Lectia 3](#) | [Lectia 4](#) | [Lectia 5](#) | [Lectia 6](#) | [Lectia 7](#)
[Material pentru laboratoare](#)

Lectia 4 - Șiruri de caractere. Aplicații

În acest curs vom discuta despre șiruri de caractere și operațiile cu șiruri de caractere. Apoi vom prezenta ideile de dezvoltare ale unor aplicații simple din lumea reală, pornind de la lucrurile învățate până acum.

Rezumat



4.1. Declararea unui șir de caractere, `char []` și `char *`

După cum am arătat și în lecția anterioară, un șir de caractere (cu capacitatea 100 caractere) se declară în C/C++ astfel:

```
char s[100];
```

De asemenea, la declararea unui șir acesta poate fi inițializat. Următoarele exemple declară șiruri de caractere și le initializează cu șirul "copil":

```
char s[11] = "copil"; // se folosesc doar 6 caractere
char t[]="copil"; // se alocă automat 6 octeți pentru șirul t:
//cele 5 litere și caracterul nul \0
char x[6]={'c','o','p','i','l','\0'}; // initializarea este similară cu
cea a unui tablou oarecare - șirurile de caractere sunt tablouri
char z[]={'c','o','p','i','l','\0'}; // se alocă automat 6 octeți pentru
șir
```

Deși o inițializare de tipul `char c[6] = "cuvant"`, în care spațiul alocat este egal cu numărul de caractere, nu va determina compilatorul să genereze un avertisment (*warning*) sau o eroare, acest lucru poate avea rezultate neașteptate dacă în memorie - la finalul șirului - nu se află (întâmplător) valoarea 0 binară. Duoă cum am spus într-o lecție anterioară, o variabilă vector conține adresa de început a vectorului (adresa primei componente a vectorului) și de aceea este echivalentă cu un pointer la tipul elementelor din vector. Deci declarațiile de mai jos vor declara fiecare câte un șir de caractere:

```
char a[6];
char *b="unsir";
char *c;
```

Diferența majoră dintre ele este însă că primele două declarații vor alocă 6 poziții în memorie, pe când ultima nu va alocă nicio zonă de memorie, necesitând să fie ulterior alocată, folosind funcțiile de alocare dinamică (`malloc()`, `calloc()`, `realloc()`). Între prima și a doua declarație diferența este mai subtilă și constă în faptul că declarația `char *b="unsir"` va determina compilatorul să plaseze șirul respectiv într-o zonă de memorie asupra căreia nu avem drepturi de scriere, deci orice încercare de a modifica acel șir va genera, cel mai probabil, o eroare de tipul *segmentation fault*.

Să considerăm o declarație similară:

```
char *p, s[31] = "programare";
```

Ce este `p`? Este un pointer la `char`, adică o variabilă a cărei valoare este adresa unei date de tip `char`. Întrebarea este a cărei date este această adresă? Deocamdată a niciuneia, deoarece variabila `p` nu a fost inițializată, iar valoarea ei este o adresă aleatorie. Șansele ca ea să reprezinte adresa unei date de tip

`char` din spațiul de memorie al programului nostru sunt la fel de mici ca șansele ca valoarea inițială a unei variabile de tip `int` să fie în intervalul 1 ... 100.

Ce este `s`? Spunem că este un șir de caractere, dar practic `s` este tot un pointer. Valoarea sa este adresa primului element din șir, adică adresa lui `s[0]`.

Observăm că de fapt, variabilele `p` și `s` sunt de același tip, pointer la `char`. Diferența dintre cele două variabile este că `s` memorează o adresa de memorie unde începe un șir de caractere (la acea adresă există o dată de tip `char`) în timp ce `p` memorează o adresă aleatorie.

Cu ce putem inițializa pointer-ul `p`? Cu adresa unei date de tip `char`. O asemenea dată este un orice element al unui șir de caractere, de exemplu orice element din `s`. Dacă `p` reprezintă adresa unui caracter dintr-un șir, atunci cu `p` se pot face toate operațiile care se pot face cu acel șir.

▼ Exemplu

```
#include <iostream>
using namespace std;
int main(){
    char * p , s[]="programare";
    cout << s << endl; // programare
    p = s;
    cout << p << endl; // programare
    p++;
    cout << p << endl; // rogramare
    return 0;
}
```

4.2. Scrierea și citirea unui șir de caractere

4.2.1. Scrierea și citirea în C++, folosind obiectele din `iostream`

- Scrierea se poate face cu operatorul `<<` de inserție în stream: `cout << s;`
- Citirea se poate face cu operatorul `>>`: `cin >> s;`

În acest mod, datorită specificului operatorului `>>` nu se pot citi șiruri care conțin spații – se vor citi caracterele până la primul spațiu, fără acesta. Pentru a citi șiruri care conțin spații, putem folosi `cin.getline()`, după cum am arătat în lecția anterioară. Se vor citi în șirul `s` caracterele din stream-ul de intrare (de la tastatură) până la apariția caracterului `'\n'` (care marchează sfârșitul de linie), dar nu mai mult de `n-1` caractere. Caracterul `'\n'` nu va fi adăugat la șirul `s`, dar va fi extras din stream. Un mic exemplu de citire a unui șir, caracter cu caracter pana la întâlnirea caracterului -:

```
#include <stdio.h>
#include <string.h>
#define N 30
int main () {
    char str[N], c;
    int n = 0;
    do {
        scanf("%c", &c);
        if (c == '\n') {
            break;
        }
        str[n++] = c;
    } while(1);
    str[n] = '\0'; // setam terminatorul de șir
    printf("%s", str);
    return 0;
}
```

4.2.2. Citirea și scrierea cu funcții din biblioteca `<stdio.h>` din C

Pentru citirea și afisarea unui șir de caractere se poate folosi flagul `'s'` la citirea cu `scanf` sau afișarea cu `printf`. De asemenea biblioteca `stdio.h` definește funcțiile `gets()` și `puts()` pentru lucrul cu șiruri de caractere.

1. **`gets(zona)`** - citește de la terminalul standard un șir de caractere terminat cu linie nouă (`<enter>`). Funcția are ca parametru adresa zonei de memorie în care se introduc caracterele citite.
Funcția returnează adresa de început a zonei de memorie unde va fi depozitat șirul citit.
2. **`puts(zona)`** - afișează la terminalul standard șirul de caractere din zona dată ca parametru, până la caracterul terminator de șir (`'\0'`), în locul căruia va afișa caracterul *sfârșit de linie*.
Funcția are ca parametru adresa zonei de memorie de unde începe afișarea caracterelor.
Funcția returnează codul ultimului caracter (diferit de `'\0'`) din șirul de caractere afișat, respectiv -1 dacă a apărut o eroare.
Funcția `gets()` va citi de la tastatură câte caractere sunt introduse, chiar dacă șirul declarat are o lungime mai mică.
Presupunem un șir declarat: `char a[]="unsir"`, care va avea deci 5 caractere. Citind un șir de lungime mai mare ca 5 de la tastatură, în șirul `a`, la afișare vom vedea ca s-a reținut tot șirul (nu doar primele 5 caractere)!. Nimic deosebit până acum. Dar dacă luăm în considerare că citirea caracterelor auxiliare se face în continuare în zona de memorie, ne punem problema ce se va suprascrie?! Răspunsul este: nu se știe... poate nimic important pentru programul nostru, poate ceva ce îl va bloca sau duce la obținerea de date eronate.
Pentru a evita aceasta, se recomandă utilizarea funcției `fgets()` pe intrarea standard `stdin`, după cum am mai precizat, într-o lecție anterioară: `fgets(zona, lung_zona, stdin)` - citește de la `stdin` un șir de caractere terminat printr-o linie nouă; dacă lungimea lui este mai mică decât `lung_zona`, sau primele `lung_zona - 1` caractere în caz contrar.
Parametrii sunt: zona de memorie, lungimea maxima admisă a șirului și terminalul standard de intrare. În cazul în care șirul dorit are lungime mai mică decât cea maximă, înaintea terminatorului de șir (`'\0'`), în zona de memorie va fi reținut și `<enter>`-ul dat (`'\n'`).

4.3. Funcții pentru șiruri de caractere

Pentru manipularea șirurilor de caractere în limbajul C se folosesc funcții declarate în fișierul <string.h>. Vom încerca să le detaliem puțin pe cele mai des folosite:

1. **strlen** - returnează lungimea șirului de caractere (de la început până la prima apariție a caracterului **NULL**).

▼ Exemplul 1

```
cout << strlen("programare"); // 10
char s[10]="copil";
cout << strlen(s); // 5
cout << strlen(s + 2); //3
```

▼ Exemplul 2

```
#include <stdio.h>
#include <string.h>

#define N 256

int main () {
    char text[N];
    printf("Introduceti un text: ");
    gets(text);
    printf("Textul are %u caractere.\n", strlen(text));
    return 0;
}
```

Ieșire:

```
Introduceti un text: just testing
Textul are 12 caractere.
```

2. **strcpy**: `strcpy( char* dest, const char* src );` - copiază caracterele din șirul aflat la adresa **src**, inclusiv caracterul nul (`'\0'`), în șirul al cărui prim element se află la adresa din **dest**. Șirul destinație va fi suprascris. Funcția asigură plasarea terminatorului de șir în șirul destinație după copiere.

Funcția returnează șirul destinație (adresa **dest**).

▼ Exemplu

```
char s[21], t[21] = "copil";
strcpy(s , "programare");
cout << s; // programare
strcpy(s , t);
cout << s; // copil
strcpy(s , t + 2);
cout << s; // pil
strcpy(s + 2 , t);
cout << s; // picopil
```

O aplicație interesantă a funcției **strcpy** este utilizarea ei pentru **eliminarea unui caracter dintr-un șir**:

```
char s[256], t[256];
int x;
// ...
//eliminarea
strcpy(t , s + x + 1);
strcpy(s + x , t);
```

Tot cu funcția **strcpy** putem realiza și **inserarea unui caracter într-un șir**:

```
char s[256], t[256];
int x;
// ...
//inserarea
strcpy(t , s + x);
strcpy(s + x + 1 , t);
s[x] = 'A'; // echivalent, *(s+x) = 'A';
```

3. **strncpy**: `char* strncpy(char *destination, const char *source, size_t num);` Este similară funcției **strcpy()**, dar în loc de a fi copiată toată sursa sunt copiate doar primele **num** caractere.

▼ Exemplu

```
#include <stdio.h>
#include <string.h>

#define N 40
```

```
int main () {
    char str1[] = "Exemplu";
    char str2[N];
    char str3[N];
    strcpy(str2, str1);
    strncpy(str3, "un sir", 2);
    str3[2] = '\0';
    printf("str1: %s\nstr2: %s\nstr3: %s\n", str1, str2, str3);
    return 0;
}
```

Ieșire:

```
str1: Exemplu
str2: Exemplu
str3: un
strcat()
```

4. **strcat**: `strcat( char* dest, const char* src );` - adaugă (concatenează) caracterele din șirul aflat la adresa `src`, inclusiv caracterul nul, la șirul al cărui prim element se află la adresa din `dest`. Șirul destinație trebuie să aibă suficientă memorie alocată pentru a acomoda șirul rezultat.

Funcția returnează adresa dest.

▼ Exemplu

```
char s[21]="pbinfo", t[21] = "copil";
strcat(s , t);
cout << s; // pbinfocopil
strcat(s , t + 2);
cout << s; // pbinfocopilpil
```

5. **strncat**: `char* strncat(char *destination, const char *source, size_t num);` - similară funcției `strcat()`, dar în loc de a fi concatenată toată sursa sunt concatenate cel mult primele `num` caractere din șirul sursă (aceasta putând fi și mai scurt).

▼ Exemplu

```
#include <stdio.h>
#include <string.h>

#define N 80

int main () {
    char str[N];
    strcpy(str, "ana ");
    strcat(str, "are ");
    strcat(str, "mere ");
    puts(str);
    strncat(str, "si pere si prune", 7);
    puts(str);
    return 0;
}
```

Ieșire:

```
ana are mere
ana are mere si pere
```

6. **strchr**: `strchr( char* str, char ch );` - caută caracterul `ch` în șirul al cărui prim caracter se află în memorie la adresa din `str`. Funcția returnează adresa `NULL`, dacă caracterul `ch` nu apare în șirul `str`, respectiv adresa primei apariții (de la stânga la dreapta) al lui `ch` în `str`, dacă `ch` apare în `str`.

▼ Exemplu

```
char s[21]="pbinfo";
char * p = strchr(s , 'i');
cout << p; // info
```

7. **strstr**: `strstr( char* s, const char* t );` - caută șirul `t` în șirul al cărui prim caracter se află în memorie la adresa din `s`. Funcția returnează adresa `NULL`, dacă șirul `t` nu apare în șirul `s`, respectiv adresa primei apariții al lui `t` în `s` (de la stânga la dreapta), dacă `t` apare în `s`.
8. **strrchr**: `char* strrchr(const char *str, int ch);` - caută caracterul `ch` în șirul `str` și returnează un pointer la ultima sa apariție (cea mai din dreapta) sau `NULL` dacă acesta nu există în șir.
9. **strcmp**: `strcmp( char* s, const char* t );` - compară lexicografic cele două șiruri de caractere:
- o dacă șirul `s` este lexicografic mai mic decât `t`, atunci funcția va returna o valoare negativă
 - o dacă șirul `s` este lexicografic mai mare decât `t`, funcția va returna o valoare pozitivă
 - o dacă cele două șiruri sunt identice funcția va returna valoarea 0

În situația în care șirurile au lungimi diferite, ultima comparație se face între `'\0'` și caracterul de pe aceeași poziție din șirul mai lung.

▼ Exemplu

```
#include <stdio.h>
#include <string.h>
```

```

#define N 80

int main () {
    char cuv[] = "rosu";
    char cuv_citit[N];
    do {
        printf ("Ghicesc culoarea...");
        gets(cuv_citit);
    } while (strcmp(cuv,cuv_citit) != 0);

    puts("OK");

    return 0;
}

```

10. **strtok**: `char *strtok( char *str, const char *sep );` - funcția are rolul de a împărți șirul `str` în *token*-uri (subșiruri separate de orice caracter aflat în lista de delimitatori `sep`), prin apelarea ei succesivă; funcția extrage din `str` câte un subșir (cuvânt, *token*) delimitat de caractere din șirul de separatori `sep`.

Funcția se apelează în două moduri: primul apel are ca parametri șirul din care se face extragerea și șirul separatorilor, iar la următoarele apeluri primul parametru este `NULL`.

Rezultatul acestei funcții este adresa de început a subșirului curent extras, respectiv `NULL` dacă nu se mai poate extrage niciun subșir din șirul dat.

Șirul din care se face extragerea se modifică în urma apelurilor. Dacă este nevoie de el mai târziu, trebuie să-i facem înainte o copie.

▼ Exemplu

Secvența de mai jos extrage dintr-un șir `s` cuvintele (separate prin caractere din mulțimea { ' ', ',', '.', ' ' }) și le afișează pe linii diferite. Șirul `s` se presupune declarat și citit.

```

char sep[]=" ., ";
char * p = strtok(s , sep);
while(p != NULL)
{
    cout << p << endl;
    p = strtok(NULL , sep);
}

```

Implementarea curentă din `<string.h>` nu permite folosirea `strtok()` în paralel pe mai mult de un șir.

▼ Exemplu

```

#include <stdio.h>
#include <string.h>

int main () {
    char str[] = "- Uite, asta e un sir.";
    char *p;
    p = strtok(str, " ,.-");
    /* separa sirul in "tokeni" si afiseaza-i pe linii separate. */
    while (p != NULL) {
        printf("%s\n", p);
        p = strtok(NULL, " ,.-");
    }

    return 0;
}

```

Ieșire:

```

Uite
asta
e
un
sir

```

11. **strdup**: `char* strdup(const char *str);` - realizează un duplicat al șirului `str`, pe care îl și returnează. Spațiul de memorie necesar copiei este alocat dinamic, fiind responsabilitatea noastră să dealocăm acea zonă de memorie.

▼ Exemplu

```

#include <stdio.h>
#include <string.h>

#define N 80

int main () {
    char str[N] = "salut", *d;

    d = strdup(str);
    if(d == NULL) {
        printf("Eroare!\n");
        return -1;
    }

    puts(d);
    free(d);

    return 0;
}

```

```
}
```

`strdup(..)` va alocă întotdeauna `strlen() + 1` octeți pentru destinație, indiferent de dimensiunea memoriei alocate pentru sursă.

12. **memset:** `void* memset(void *ptr, int val, size_t num);` - în zona de memorie dată de pointerul `ptr` sunt setate primii `num` octeți la valoarea dată de `val`. Aceasta este o funcție mai generală, dar pentru șiruri de caractere - în care fiecare element ocupă un octet - apelul acestei funcții duce la înlocuirea primelor `num` valori cu cea dată ca argument.

Funcția returnează șirul `ptr`.

▼ Exemplu

```
#include <stdio.h>
#include <string.h>

int main () {
    char str[] = "nu prea vreau vacanta!";
    memset(str, '-', 7);
    puts(str);
    return 0;
}
```

Ieșire:

```
----- vreau vacanta!
```

Observație: Ca programatori, veți întâlni adesea situații în care `memset` este folosit pentru inițializarea de vectori de diverse tipuri. Atunci când scrieți sau evaluați cod care face acest lucru trebuie să aveți în vedere că `memset` face scrierea valorii primite pe fiecare octet (fără a ține cont de dimensiunea reprezentării tipului de date).

Următorul cod arată cum putem folosi `memset` pentru a inițializa un vector de `int` la 0 folosind `memset`, respectiv care ar fi rezultatul dacă am încerca să inițializăm vectorul cu o altă valoare:

```
#include <stdio.h>
#include <string.h>

#define SIZE 2

int main(void)
{
    int a[SIZE], b[SIZE], i;
    memset(a, 0, SIZE * sizeof(int));
    memset(b, 5, SIZE * sizeof(int));

    for(i = 0; i < SIZE; i++)
        printf("%d ", a[i]);

    printf("\n");

    for(i = 0; i < SIZE; i++)
        printf("%d ", b[i]);

    return 0;
}
```

Rezultatul este

```
...
0 0
84215045 84215045
```

Rezultatul neașteptat de pe a doua linie provine din faptul că fiecare octet al `int`-urilor a fost setat la 5 și nu valoarea întregii structuri. Este de menționat faptul că utilizarea `memset` în astfel de situații nu este recomandată.

13. **memmove:** `void* memmove(void *destination, const void *source, size_t num);` - copiază un număr de `num` caractere de la sursă la zona de memorie indicată de destinație. Copierea are loc ca și cum ar exista un buffer intermediar, deci sursa și destinația se pot suprapune. Funcția nu verifică terminatorul de șir la sursă, copiază mereu `num` octeți, deci, pentru a evita depășirea, trebuie ca dimensiunea sursei să fie mai mare ca `num`.

Funcția returnează destinația.

▼ Exemplu

```
#include <stdio.h>
#include <string.h>

int main () {
    char str[] = "memmove can be very useful.....";
    memmove(str + 20, str + 15, 11);
    puts(str);
    return 0;
}
```

Ieșire:

memmove can be very very useful.

14. memcpy

```
void* memcpy(void *destination, const void *source, size_t num);
```

Copiază un număr de num caractere din șirul sursă în șirul destinație. Funcția returnează șirul destinație.

▼ Exemplu

```
#include <stdio.h>
#include <string.h>

#define N 40

int main () {
    char str1[] = "Exemplu";
    char str2[N];
    char str3[N];
    memcpy(str2, str1, strlen(str1) + 1); // + 1 este necesar pentru a copia și terminatorul de șir
    memcpy(str3, "un sir", 7);
    printf("str1: %s\nstr2: %s\nstr3: %s\n", str1, str2, str3);
    return 0;
}
```

Ieșire:

```
str1: Exemplu
str2: Exemplu
str3: un sir
```

Bibliografie

<https://ocw.cs.pub.ro/courses/programare/laboratoare/lab10> https://profs.info.uaic.ro/~infogim/2017/lectii/78/783_stringuri.pdf
<http://curs.algoritmi.ro/2012/09/30/pc-siruri/> <http://www.cplusplus.com/reference/cstring/> <http://www.cplusplus.com/reference/string/string/>
<http://www.pbinfo.ro/?pagina=articole&subpagina=afisare&id=19> <http://campion.edu.ro/arhiva/index.php>

4.4. Lista de aplicații și idei de rezolvare

1. Aplicația 1: Despărțirea în silabe a unui cuvânt, după regulile din limba română

Scrieți o funcție care desparte un cuvânt din limba română în silabe.

▼ Idee de rezolvare

Vom lua regulile de despărțire în silabe de la <https://www.limba-romana.net/lectie/Reguli-de-despartire-a-cuvintelor-in-silabe/72/>. Vom aplica aceste reguli, pornind de la cea mai particulară la cea mai generală. Dar mai întâi vom verifica dacă nu cumva suntem într-un caz de excepție de la reguli. Una dintre aceste reguli este: „Două vocale în hiat se despart între ele, făcând parte din silabe diferite: a-er; a-le-e; po-e-zi-e;” Deoarece noi vom face o prelucrare pe șiruri de caractere, vom lucra cu litere și nu cu sunete. Astfel, unele grupuri de două-trei vocale vor putea fi interpretate atât ca vocale în hiat, cât și ca diftong (respectiv triftong). De pildă, la întâlnirea grupului de vocale **ie** nu putem ști dacă ele trebuie despărțite (hiat ca în **sanie** - > **sa-ni-e**) sau nu (diftong ca în **ploaie** -> **ploa-ie**). La adresa http://www.etc.tuiasi.ro/sibm/romanian_spoken_language/ro/hiat.htm se găsesc explicații și exemple despre hiat. Din punct de vedere al șirurilor de caractere, regula de mai sus poate genera probleme. Dacă aplicarea ei nu ar duce la rezultate corecte, atunci ar fi bine să fie stocate întâi excepțiile, sub forma de perechi cuvânt - cuvânt despărțit în silabe. Abia după testarea excepțiilor, se vor putea aplica regulile de mai sus. Se poate începe cu regula cea mai strictă, iar la fiecare regulă cu excepțiile (de exemplu cu excepțiile de la regula 3).

2. Aplicația 2: Convertirea unui număr din cifre în litere

Scrieți o funcție care codifică un număr natural între 0 și 100000000 într-un șir de caractere, reprezentând numărul scris cu litere, în limba română. De exemplu, pentru numărul 70054 rezultatul funcției va fi „saptezecideciincizecicisipatru”.

▼ Idee de rezolvare

Este o aplicație ușoară și sperăm că va fi abordată de toți studenții. Atenție la numerele care se exprimă prin feminin versus masculin, precum douăzeci (nu doizeci), două sute (nu doisute) sau la excepțiile de la 4 și 6: spunem paisprezece și șasezeci, nu patrusprezece sau șasezeci. De asemenea, mare atenție la numerele care conțin zerouri în interior, de exemplu: 10040, care este zecemii patruzeci, nu zecemii zerosute patruzeci. **Propunere:** Extindeți programul și pentru limba franceză, unde lucrurile sunt mai complicate. De exemplu, pentru a exprima 94 scriem ceva de genul quatre-vingt-quatorze, adică 4 de 20 și 14.

3. Aplicația 3: Codificare ADN

Codificați numerele întregi folosind acidul dezoxiribonucleic (ADN), ce conține A (adenină), C (citozină), G (guanină) și T (timină). Astfel, în calcule se folosește baza de numerație 4, dar cifrele A, C, G și T au valorile A=0, C=1, G=2, iar T=-1. De exemplu, numărul întreg 27 se reprezintă, folosind ADN, prin "GTT". Asemănător, numărul întreg -6 se reprezintă prin "TGG", iar numărul 84 se reprezintă prin "CCCA".

▼ Idee de rezolvare

Vom considera declarațiile:

```
#define MAX_ADN 10
typedef char sirADN[MAX_ADN];
```

Vom codifica un număr întreg folosind acidul dezoxiribonucleic (ADN), ce conține A (adenină), C (citozină), G (guanină) și T (timină), folosind în calcule baza de numerație 4, dar cifrele A, C, G și T vor avea valorile A=0, C=1, G=2, iar T=-1. De exemplu, numărul întreg 27 se reprezintă, folosind ADN, prin "GTT". Asemănător, numărul întreg -6 se reprezintă prin "TGG", iar numărul 84 se reprezintă prin "CCCA". Justificare:

- GTT = $G \times 4^2 + T \times 4^1 + T \times 4^0 = 2 \times 16 + (-1) \times 4 + (-1) \times 1 = 32 - 4 - 1 = 27$,
- TGG = $T \times 4^2 + G \times 4^1 + G \times 4^0 = (-1) \times 16 + 2 \times 4 + 2 \times 1 = -16 + 8 + 2 = -6$, CCCA = $C \times 4^3 + C \times 4^2 + C \times 4^1 + A \times 4^0 = 1 \times 64 + 1 \times 16 + 1 \times 4 + 0 \times 1 = 84$

Vom scrie funcții pentru fiecare operație de codificare/decodificare/adunare/scădere etc. Pentru a realiza decodificarea, vom parcurge șirul de la dreapta la stânga, înmulțind valoarea corespunzătoare literei cu puterea lui 4 la care am ajuns (pornind cu $1 = 4^0$). De fiecare dată adunăm acest produs la o sumă inițial zero.

Pentru a face codificarea, va trebui să facem un ciclu în care să vedem cât este restul împărțirii numărului la 4. Acesta poate fi 0, 1, 2 sau 3. Evident, pentru 0, 1 sau 2 putem scrie „cifra” (litera ACGT) corespunzătoare, dar trebuie să ne gândim cum facem când restul împărțirii este 3. În acest caz, vom folosi un artificiu.

4. Aplicația 4: Calculator în limba română

Programul va putea face calcule simple de tipul `operand1 operator operand2`, în care operatorii sunt cei aritmetici (+, -, *, /), plecând de la o propoziție interogativă în limba română, de genul "Cât este suma dintre 3 și patru?" sau "Care este rezultatul înmulțirii lui zece cu paisprezece?".

▼ Idee de rezolvare

Algoritmul: pentru a rezolva această problemă, vom lua un exemplu: „Care este rezultatul înmulțirii lui zece cu paisprezece?” se desparte propoziția în cuvinte, apoi se execută următoarele cuvintele inutile se elimină (de exemplu, „Care”, „este”, „rezultatul”, „lui”, „cu”); celelalte cuvinte se aduc la o formă standard (de exemplu, „înmulțirii” -> „*”, „zece” -> „10”, „paisprezece” -> „14”); vectorul de trei cuvinte obținut, se pune în ordine infixată (de exemplu, [„10”, „*”, „14”]); se evaluează expresia, transformând șirurile de cifre în numere și ținând cont de operator; astfel se obține o valoare numerică (de exemplu, 140); se transformă valoarea numerică în cuvânt în limba română (de exemplu, „o sută patru zeci”).

5. Aplicația 5 - Corectitudinea unei expresii algebrice

Se va verifica dacă o expresie algebrică este corectă, conform regulilor de sintaxă, dintr-un limbaj de programare. De exemplu, expresiile $(x+3/2-(y*\sin(z)))$ și $2+1/\cos(\ln(x))$ sunt corecte, dar $\cos(3(\sin(1x))$ și $x+3-$ nu sunt corecte în limbajul C.

▼ Idee de rezolvare

Prima dată vom despărți șirul în token-uri, fiecare token putând fi o variabilă (litere/cifre/_ , începând cu literă), constante (cifre/.), nume de funcții (sin/cos/ln etc.), operatori (simbolurile +, -, *, /). Se vor verifica dacă sunt îndeplinite următoarele condiții:

- Numărul de paranteze închise este egal cu numărul de paranteze deschise.
- La o parcurgere a șirului, de la stânga la dreapta, până la orice poziție trebuie ca numărul de paranteze deschise să fie mai mare sau egal cu cel al parantezelor închise.
- După un operator poate urma o variabilă (deci o literă), o funcție (deci sin/cos/ln etc.), o constantă (deci o cifră) sau o paranteză deschisă (().
- După o variabilă poate urma un operator sau o paranteză închisă ()).
- După o constantă poate urma un operator sau o paranteză închisă ()).
- După un nume de funcție poate urma doar o paranteză deschisă (().
- După paranteză deschisă (()) poate urma o altă paranteză deschisă, un nume de funcție, o variabilă, o constantă sau chiar operatorul unar + sau -.
- După paranteză închisă ()) poate urma o altă paranteză închisă sau un operator.